IFA SkillQuest Platform - Design Document

1. Project Overview

IFA SkillQuest is a web-based hiring platform featuring gamified cognitive assessments for candidate evaluation. The platform provides sequential game-based testing with automated scoring, role-based dashboards, and comprehensive analytics.

Tech Stack:

-Frontend: React 18 + TypeScript + Vite + TailwindCSS
-Backend: Node.js + Express.js
-Database: MongoDB Atlas (Mongoose ODM)
-Authentication: Google OAuth 2.0
-Deployment: Vercel (frontend + serverless backend)

2. Goals & Non-Goals

Goals

- Provide gamified cognitive assessment for job applicants
- Implement sequential game unlocking with anti-cheating measures
- Generate automated scoring and leaderboard ranking
- Support multi-role authentication (Applicant, Admin)
- Track detailed metrics per candidate

Non-Goals

- Real-time video proctoring
- Integration with external ATS systems
- Employee/Client role full implementation (placeholders only)
- Mobile native app support

3. High-Level Architecture

User

↓ HTTPS

React SPA (Frontend)

- Authentication
- Game UI
- Dashboards

↓ REST (JSON)

Express API (Backend)

- Controllers
- Routes
- Middleware

↓ CRUD

MongoDB Atlas

- Users
- Profiles
- Assessments

Communication

RESTful API with JSON payloads

Authentication

JWT-based session management via Google OAuth

CORS

Configured for cross-origin requests between frontend and backend

4. System Components

4.1 Frontend (React + TypeScript)

Core Modules:

Authentication: Google OAuth integration with role-based routing

Game Engines:

- Unblock Me (sliding block puzzles)
- Minesweeper (mine detection game)
- Water Capacity (liquid transfer puzzles)
- Question Game (quiz with image support)

Dashboards:

- Applicant Dashboard (game progress, scores)
- Admin Dashboard (candidate management, analytics)
- **Anti-Cheating:** Tab switch detection with 3-strike disqualification
- **Contexts:** AuthContext for global user state

Key Libraries:

- `@react-oauth/google` - OAuth authentication
- `framer-motion` - UI animations
- `sonner` - Toast notifications
- `lucide-react` - Icons

4.2 Backend (Node.js + Express)

Architecture: Serverless-compatible Express API

Routes:

- `/api/users` - User authentication and management
- `/api/profiles` - Candidate profile CRUD
- `/api/assessments` - Game score submissions
- `/api/leaderboard` - Ranking and scoring
- `/api/settings` - Admin configuration
- `/api/question-game` - Quiz content management
- `/api/question-bank` - Question repository

Middleware:

- CORS handler with whitelist
- JSON body parser
- Global error handler

Controllers:

- `assessmentController` - Score persistence
- `leaderboardController` - Ranking logic
- `profileController` - Profile management
- `userController` - User operations

5. Database

Type: MongoDB Atlas (Cloud-hosted NoSQL)
ODM: Mongoose with schema validation

Connection Strategy:

- Connection pooling (maxPoolSize: 10)
- Serverless-compatible (bufferCommands: false)
- Graceful fallback if DB unavailable

6. Data Model Design

6.1 User Schema

javascript {
  id: String (unique, custom ID),
  googleId: String (unique),
  email: String (unique),
  name: String,
  picture: String,
  role: Enum ['applicant', 'admin', 'employee', 'client'],
  createdAt: Date
}

6.2 Profile Schema

javascript {
  userId: String (unique),
  candidateId: String (unique, format: IFA-{timestamp}-{random}),
  name: String,
  email: String,
  phone: String,
  collegeName: String,
  cgpa: String,
  location: String,
  interestedRoles: [String],
  telegramId: String,
  resumeLink: String,
  websiteLink: String,
  profileCompleted: Boolean,
  createdAt: Date
}

6.3 Assessment Schema

```
javascript {
  userId: String (unique),
  candidateId: String,
  games: {
    minesweeper: Object,
    'unblock-me': Object,
    'water-capacity': Object,
    'question-game': Object
  },
  totalScore: Number,
  completedAt: Date,
  trialMode: {
    minesweeper: Boolean,
    'unblock-me': Boolean,
    'water-capacity': Boolean
  }, createdAt: Date
}
```

6.4 Game Object Structure:

```
javascript {
  score: Number,
  timeSpent: Number (seconds),
  levelsCompleted: Number,
  moves: Number,
  errors: Number,
  failed: Boolean,
  failureReason: String,
  completedAt: String (ISO timestamp)  }
```

7. Core Workflows

7.1 User Registration & Authentication

1. User selects role (Applicant/Admin) on landing page
2. Google OAuth consent screen initiated
3. Backend validates email against admin whitelist (if role=admin)
4. New user created or existing user retrieved
5. JWT session token stored, user redirected to role-specific dashboard

7.2 Profile Completion (Applicant)

1. User fills mandatory profile fields
2. Resume upload (optional)
3. Candidate ID auto-generated: `IFA-{timestamp}-{random}`
4. Profile saved to database
5. Assessment dashboard unlocked

7.3 Game Assessment Flow

Sequential Unlocking:

1. **Unblock Me** - Always unlocked first
2. **Minesweeper** - Unlocks after Unblock Me completed/failed
3. **Water Capacity** - Unlocks after Minesweeper completed/failed
4. **Question Game** - Unlocks after all previous games completed/failed

Game Execution:

1. User clicks "Start Assessment" (5-minute timer begins)
2. Tab switch detection active (3 violations = disqualification)
3. Game runs for 5 minutes or until manual submission
4. Score calculated and submitted to backend
5. Next game unlocks, user returned to dashboard

Trial Mode:

- Non-scored practice version of each game
- No time limit
- Unlocks only after scored version completed
- Does not affect assessment scores

7.4 Scoring & Submission

1. Game completion triggers score calculation
2. Frontend sends POST to `/api/assessments` with game data
3. Backend performs upsert on Assessment document
4. Total score recalculated using weighted formula
5. Leaderboard updated with new ranking

7.5 Admin Workflow

1. Admin views candidate table with filters
2. Admin clicks candidate to view detailed scorecard
3. Admin accesses leaderboard with sortable columns
4. Admin can upload/manage question banks (Excel/JSON)

8. Scoring & Ranking Logic

8.1 Total Score Formula

Total Score = (Unblock Me × 6) + (Minesweeper × 5) +
(Water Capacity × 3) + (Quiz Score × 10)

Multipliers Rationale:

- Unblock Me (×6) - Highest cognitive load
- Minesweeper (×5) - High volume, lower individual complexity
- Water Capacity (×3) - Moderate difficulty
- Quiz (×10) - Scales percentage to comparable range

8.2 Leaderboard Ranking

Primary Sort: Total Score (descending)

Tie-Breaker: Total Time Spent (ascending)

Implementation:

javascript

```
leaderboard.sort((a, b) => {
  if (b.totalScore !== a.totalScore) {
    return b.totalScore - a.totalScore;
  }
  return a.totalTimeSpent - b.totalTimeSpent;
});
```

8.3 Individual Game Scoring Examples

Unblock Me:

- Base score per level: 5 points
- Deduct 1 point per move over optimal
- Minimum 1 point per level
- Completion bonus: 200 points for all 10 levels

Minesweeper:

- Score = levels completed within 5 minutes
- Errors tracked but don't reduce score (used for analytics)

Water Capacity:

- Score = puzzles solved within 5 minutes
- Steps tracked for optimal solution analysis

9. Security Design

9.1 Authentication

- **Google OAuth 2.0:** Trusted identity provider
- **Admin Whitelist:** Hardcoded email validation for admin role
- **Session Management:** User data stored in localStorage with JWT-like structure

9.2 Anti-Cheating Measures

Tab Switch Detection

- Visibility API monitors focus changes
- 3 violations → automatic disqualification
- Score set to 0, game marked as failed

Time Enforcement: 5-minute hard limit per game

Trial Mode Separation: Non-scored trials don't affect rankings

9.3 API Security

- **CORS Whitelist:** Only allowed origins can access API
- **Input Validation:** Mongoose schema validation on all writes
- **Error Handling:** Centralized error middleware prevents info leakage

9.4 Data Privacy

- Google OAuth scopes limited to basic profile
- Resume files stored as links (not binary in DB)
- No PII logged in console/errors

10. Scalability Considerations

10.1 Database

- **MongoDB Atlas:** Auto-scaling storage and throughput
- **Indexed Fields:** userId, candidateId, email, googleId
- **Connection Pooling:** Max 10 concurrent connections per serverless instance

10.2 Backend

- **Serverless Architecture:** Deployed on Vercel Functions
- **Stateless API:** No in-memory session storage
- **Horizontal Scaling:** Automatic under Vercel's infrastructure

10.3 Frontend

- **Static Asset CDN:** Vite build output served via Vercel Edge Network
- **Code Splitting:** Route-based lazy loading
- **Client-Side Caching:** localStorage for user state

10.4 Current Limitations

- **Rate Limiting:** Not implemented (future enhancement)
- **CDN for Uploads:** Files stored on backend server (consider S3 for production)
- **Database Sharding:** Not required at current scale

11. Failure Handling & Edge Cases

11.1 Network Failures

Database Connection Loss:

- Graceful degradation with error logs
- API returns 500 with generic message
- **API Timeout:** Frontend shows toast notification, allows retry
- **Resume Upload Failure:** Optional field, doesn't block profile completion

11.2 Authentication Failures

- **Invalid Admin Email:** User redirected with error toast
- **OAuth Denial:** User returned to landing page
- **Role Mismatch:** Existing user attempting to login with different role rejected

11.3 Game Interruptions

- **Tab Switch (3x):** Automatic disqualification, score = 0
- **Timer Expiration:** Current progress saved, game marked complete
- **Browser Crash:** No recovery mechanism (assessment lost)

11.4 Data Integrity

- **Duplicate Submissions:** Mongoose unique constraints prevent duplicates
- **Invalid Score Data:** Mongoose validation rejects malformed payloads
- **ID Collisions:** UUID-based generation makes collisions statistically improbable

11.5 Admin Operations

- **Concurrent Edits:** Last-write-wins (no optimistic locking)
- **Bulk Upload Errors:** Validation failures logged, partial success allowed
- **Leaderboard Caching:** Computed on-demand (no stale data risk)

11.6 Known Edge Cases

- **Incomplete Profiles:** Assessment locked until profile completed
- **Multiple Role Accounts:** Same Google account cannot have multiple roles
- **Trial Mode Before Assessment:** Trial modes locked until scored version completed
- **Refresh During Game:** Timer/progress not persisted (user must restart)

Appendix: Key Files Reference

Backend:

- `backend/server.js` - Main Express server
- `backend/server/models/` - Mongoose schemas
- `backend/server/controllers/` - Business logic
- `backend/server/routes/` - API endpoints

Frontend:

- `frontend/src/components/games/` - Game implementations
- `frontend/src/contexts/AuthContext.tsx` - Authentication logic
- `frontend/src/hooks/useTabSwitchDetection.ts` - Anti-cheat system
- `frontend/src/lib/utils.ts` - Score calculation functions

Last Updated: January 10, 2026

Maintained By: IFA SkillQuest Development Team